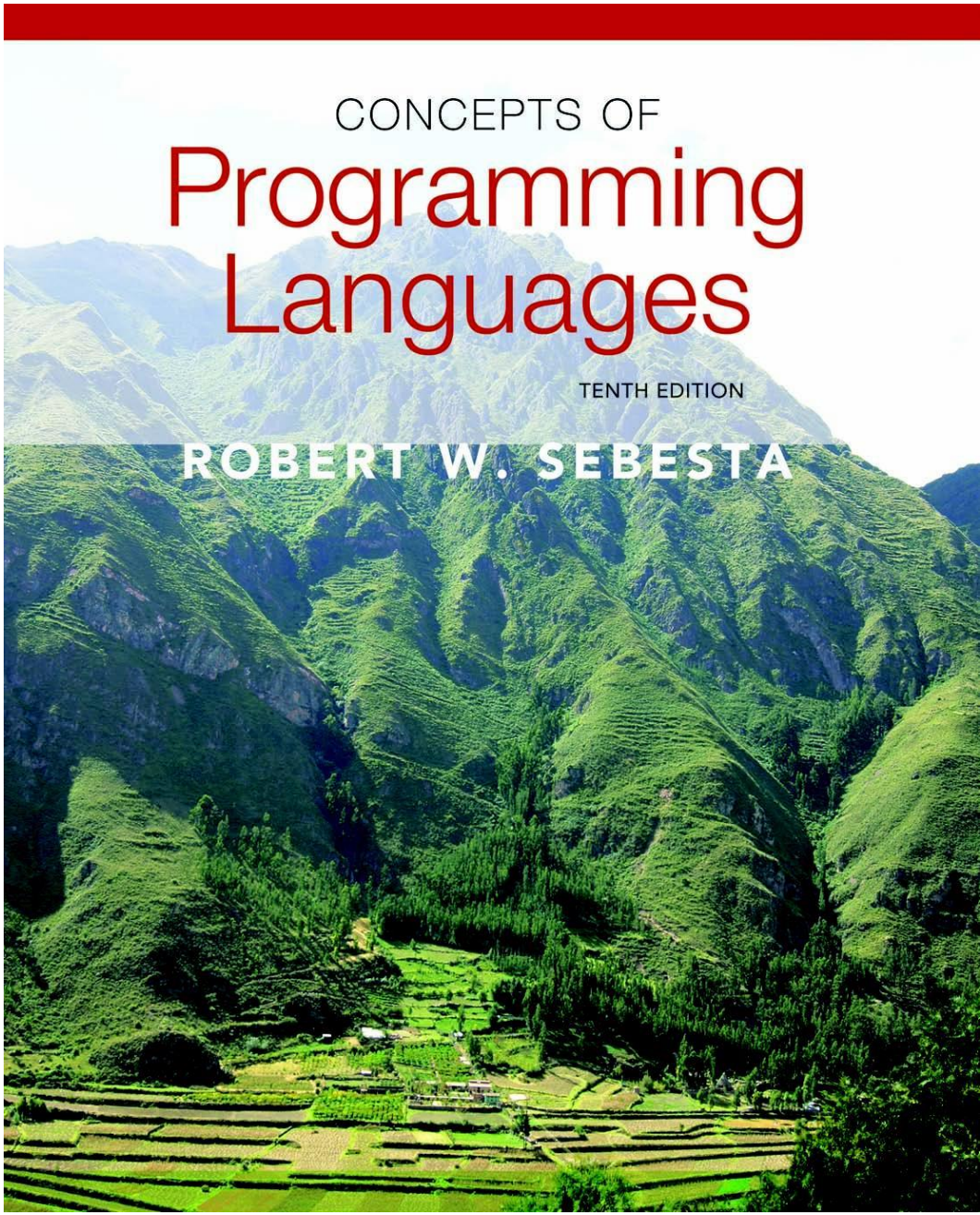
Chapter 15

Functional Programming Languages



Chapter 15 Topics

- Introduction
- Mathematical Functions
- Fundamentals of Functional Programming Languages
- Support for Functional Programming in Primarily Imperative Languages
- Comparison of Functional and Imperative Languages

Copyright © 2012 Addison-Wesley. All rights reserved.

Introduction

- The design of the imperative languages is based directly on the *von Neumann architecture*
  - Efficiency is the primary concern, rather than the suitability of the language for software development
- The design of the functional languages is based on *mathematical functions*
  - A solid theoretical basis that is also closer to the user, but relatively unconcerned with the architecture of the machines on which programs will run

Copyright © 2012 Addison-Wesley. All rights reserved.

Mathematical Functions

- A mathematical function is a *mapping* of members of one set, called the *domain set*, to another set, called the *range set*
- A *lambda expression* specifies the parameter(s) and the mapping of a function in the following form

  `λ(x) x * x * x`

  for the function `cube(x) = x * x * x`

Copyright © 2012 Addison-Wesley. All rights reserved.

Lambda Expressions

- Lambda expressions describe nameless functions
- Lambda expressions are applied to parameter(s) by placing the parameter(s) after the expression

  e.g., `(λ(x) x * x * x)(2)`

  which evaluates to `8`

Copyright © 2012 Addison-Wesley. All rights reserved.

Functional Forms

- A higher-order function, or *functional form*, is one that either takes functions as parameters or yields a function as its result, or both

Copyright © 2012 Addison-Wesley. All rights reserved.

Function Composition

- A functional form that takes two functions as parameters and yields a function whose value is the first actual parameter function applied to the application of the second

  Form: `h ≡ f ° g`

  which means `h (x) ≡ f ( g ( x))`

  For `f (x) ≡ x + 2` and `g (x) ≡ 3 * x`,

  `h ≡ f ° g` yields `(3 * x)+ 2`

Copyright © 2012 Addison-Wesley. All rights reserved.

Apply-to-all

- A functional form that takes a single function as a parameter and yields a list of values obtained by applying the given function to each element of a list of parameters

  Form: `αh(x) ≡ x * x`

  For `α(h, (2, 3, 4))` **yields** `(4, 9, 16)`

Copyright © 2012 Addison-Wesley. All rights reserved.

Fundamentals of Functional Programming Languages

- The objective of the design of a FPL is to mimic mathematical functions to the greatest extent possible
- The basic process of computation is fundamentally different in a FPL than in an imperative language
  - In an imperative language, operations are done and the results are stored in variables for later use
  - Management of variables is a constant concern and source of complexity for imperative programming
- In an FPL, variables are not necessary, as is the case in mathematics
- *Referential Transparency* – In an FPL, the evaluation of a function always produces the same result given the same parameters

Copyright © 2012 Addison-Wesley. All rights reserved.

Primitive Function Evaluation

- Parameters are evaluated, in no particular order
- The values of the parameters are substituted into the function body
- The function body is evaluated
- The value of the last expression in the body is the value of the function

Copyright © 2012 Addison-Wesley. All rights reserved.

Haskell

- A static-scoped functional language
- Uses type declarations, but also does *type inferencing* to determine the types of undeclared variables
- It is *purely* functional

Ex1: let fact n = if n<=1 then 1 else n * fact (n - 1)

Ex2: let fib n = if n<=1 then n else fib (n-1)+ fib(n-2)

Copyright © 2012 Addison-Wesley. All rights reserved.

Function Definitions with Different Parameter Ranges

```
sub n
  | n < 10      = 0
  | n > 100     = 2
  | otherwise   = 1
```

```
let sub n = if n<10 then 0 else if n>100 then 2 else 1
```

- Because Haskell support polymorphism, this works for any numeric type of `x`

Copyright © 2012 Addison-Wesley. All rights reserved.

Haskell Lists

- List notation: Put elements in brackets
  e.g., `directions = ["north", "south", "east", "west"]`
- Length: `length`
  e.g., `length directions` is `4`
- Arithmetic series with the `..` operator
  e.g., `[2, 4..10]` is `[2, 4, 6, 8, 10]`

Copyright © 2012 Addison-Wesley. All rights reserved.

Haskell Lists

- Catenation is with `++`
  e.g., `[1, 3] ++ [5, 7]` results in `[1, 3, 5, 7]`
- Colon operator
  e.g., `1:[3, 5, 7]` results in `[1, 3, 5, 7]`
- Head and tail
  e.g., `head [3, 5, 7]` results in `3`
  e.g., `tail [3, 5, 7]` results in `[5, 7]`

Copyright © 2012 Addison-Wesley. All rights reserved.

Haskell

- Pattern Parameters

```
productList :: [Int] -> Int
productList []     = 1
productList (a:x)  = a * productList x
```

  - Factorial:

```
fact :: Int -> Int
fact n = productList [1..n]
```

- List Comprehensions (Chapter 6)

```
cubes :: Int -> [Int]
cubes n = [x^3 | x <- [1..n]]
```

- List Comprehensions with condition

```
factors :: Int -> [Int]
factors n=[i|i<-[1..n `div` 2],n `mod` i == 0]
```

The backticks specify the function is used as a binary operator

Copyright © 2012 Addison-Wesley. All rights reserved.

Quicksort

```
sort :: (Ord a) => [a] -> [a]
sort [] = []
sort (h:t) =
    sort [b | b <- t, b <= h]
      ++
    [h]
      ++
    sort [b | b <- t, b > h]
```

Illustrates the concision of Haskell

Copyright © 2012 Addison-Wesley. All rights reserved.

Lazy Evaluation

- A language is *strict* if it requires all actual parameters to be fully evaluated
- A language is *nonstrict* if it does not have the strict requirement
- Nonstrict languages are more efficient and allow some interesting capabilities – *infinite lists*
- Lazy evaluation – Only compute those values that are necessary

Ex: Square numbers

```
squares :: [Integer]
squares = [n * n | n <- [0..]]
```

Determining if `16` is a square number

```
member :: Eq a => [a] -> a -> Bool
member [] _ = False
member (a:x) b = (a == b) || member x b
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Comparing Functional and Imperative Languages

- Imperative Languages:
  - Efficient execution
  - Complex semantics
  - Complex syntax
  - Concurrency is programmer designed
- Functional Languages:
  - Simple semantics
  - Simple syntax
  - Less efficient execution
  - Programs can automatically be made concurrent

Copyright © 2012 Addison-Wesley. All rights reserved.